Gemma-RL: Technical Report

Building a Gemma-4-31B software-engineering agent for the Kaggle *Gemma 4 Developer Agent* competition, and researching learned (RL) graph traversal for fault localization

Project team (prepared with Claude Code)

Status as of 2 October 2026

Abstract

This report documents everything done so far in the Gemma-RL project: the problem and its constraints, the data, the evaluation infrastructure we had to build (a Docker-free reproduction of the official harness, model serving on two shared 24 GB GPUs with a patched vLLM, and a Kaggle evaluation pipeline on the scorer's real hardware), the fault-localization research (ground-truth labelling, BM25 / embedding / learning-to-rank baselines under leave-one-repository-out validation, graph smoothing, and a deployed BM25 locator skill), eight end-to-end agent configurations (C1–C6) evaluated on the scorer hardware with failure triage, the measurement protocol, the groundwork for fine-tuning (LoRA loading, exact-format supervised data rendering, compaction-aware dataset construction), new training data (66 verified tasks mined from git history, 1,628 synthetic localization bugs with LLM-written issues), and the formal design of the reinforcement-learning traverser. Every number in this report comes from files in the repository; the methodology is given with both intuition and the underlying mathematics.

Contents

1 Executive summary **3**

2 Problem setting **3**
2.1 The task . . . 3
2.2 Constraints that shaped every decision . . . 3

3 Data **4**
3.1 Audit . . . 4

4 Infrastructure **4**
4.1 Three Python environments . . . 4
4.2 A Docker-free reproduction of the scorer . . . 4
4.3 Serving a 31B model on two shared 24 GB GPUs . . . 5
4.4 Evaluation on the scorer's hardware . . . 6

5 Fault localization research **6**
5.1 Ground-truth labels from gold patches (B1) . . . 6
5.2 Anchors . . . 6
5.3 R1: BM25 . . . 6
5.4 R2: embedding pseudo-query . . . 7
5.5 R3: learning to rank (LambdaMART) . . . 7
5.6 R3+G: graph smoothing . . . 7
5.7 Metrics . . . 7
5.8 Results . . . 7
5.9 The deployed locator skill . . . 8

6 The agent (scaffold) and end-to-end experiments **9**
6.1 How the harness drives an agent . . . 9
6.2 Configurations . . . 9
6.3 Results on the scorer's hardware . . . 9
6.4 Four failure mechanisms we found and fixed . . . 10
6.5 How much of a difference is real? Noise, pass@k and stochastic tasks . . . 10

7 Measurement protocol **11**

8 Fine-tuning groundwork **11**
8.1 LoRA on the quantized checkpoint . . . 11
8.2 Supervised objective . . . 12
8.3 Rendering trajectories in the exact training format . . . 12
8.4 Context compaction and segmentation . . . 12

9 New training data **13**
9.1 Tasks mined from git history (B4) . . . 13
9.2 Our own code-graph builder . . . 13
9.3 Synthetic localization bugs (B5) . . . 13

10 The reinforcement-learning traverser (designed, data-gated) **14**
10.1 MDP . . . 14
10.2 Policy and learning . . . 14
10.3 Why it is gated, and how it would ship . . . 14

11 Lessons learned **14**

12 Status and next steps **15**

A Where things are **15**

B Reproducing the main results **15**

1 Executive summary

Area	Result
Competition	129 public tasks (fastapi 67, rich 48, requests 13, httpx 1); hidden set ≈120 tasks; 12 h runtime; 1 submission/day; model fixed to `gemma-4-31b-it-qat-w4a16-ct`.
Local harness	Docker-free, faithful to the scorer's verification; **107/129** tasks are locally *valid* (gold patch passes, empty patch fails).
Serving	31B W4A16 model served on 2 shared A5000s via an uneven pipeline split (52/8 layers), 4 vLLM patches, ≈21.5 tok/s per stream.
Localization (LORO)	BM25 node MRR **0.398**; supervised LambdaMART ranker 0.397±0.012; graph smoothing 0.385; embedding pseudo-query 0.112. With 3 training repositories the learned ranker does *not* beat BM25.
Locator skill	File hit@1 **0.49**, hit@3 0.69 (public best known: 0.43 / 0.70@5), <1 s per call.
Agent (scorer HW, same 30 tasks)	Best configurations solve 9–10/30 (30–33%); per-run std ≈0.024; 8 runs jointly solve **14/30**: pass@8 = 0.467 vs pass@1 = 0.29.
Key failure modes fixed	malformed skill calls (backticks), test-file edits that break hidden-test application (0 after fix), thinking-driven timeouts, empty submissions under tight tool budgets.
Submissions	C4 submitted (pending); C6a (same accuracy, 34% faster) frozen as `scaffold-v1-frozen`, next submission.
Fine-tuning groundwork	LoRA (rank 16, 122M params, 233.5 MiB) loads on the QAT checkpoint and changes outputs; SFT rendering reproduces vLLM's prompts token-exactly on 15/16 turns.
New data	66 verified mined tasks; 1,628 injected bugs in 22 packages; own graph builder with node parity 1.00 on non-test code.

Table 1: One-page summary of results.

Intuition. The model already *can* solve far more tasks than it solves in any single attempt (pass@8 ≫ pass@1). That gap is the central quantitative fact of the project so far: it makes rejection-sampling fine-tuning (train on the model's own successes) the main competition lever, while better localization matters less than expected because most failures are "right file, wrong fix".

2 Problem setting

2.1 The task

Each task t is a real GitHub pull request from one of four Python repositories, given as the tuple

$$(\text{repo},\ \text{base commit},\ \text{problem statement},\ \text{hints},\ \text{gold patch } p^\star,\ \text{test patch}).$$

The agent sees only the repository at `base_commit` and the problem statement (hints are empty for all 129 public tasks). It must produce a patch p. Scoring happens in a fresh sandbox: apply p, reset any test/config files, apply the hidden `test_patch`, run `pytest` on the targeted test files.

$$R(t,p) = \mathbb{1}\big[\text{pytest exits } 0 \ \wedge\ \text{passed} > 0 \ \wedge\ \text{failures} = \text{errors} = 0 \ \wedge\ \text{all required tests passed}\big],$$
$$\text{Score} = \frac{1}{|T|}\sum_{t\in T} R(t, p_t).$$

2.2 Constraints that shaped every decision

- **Fixed model:** all agents must use `gemma-4-31b-it-qat-w4a16-ct` (31B parameters, weights quantized to 4 bit with group size 32, activations 16 bit), served by vLLM with tensor parallelism 4 on 4×L4 (24 GB each), `max_model_len`=32,768, `gpu_memory_utilization`=0.80.
- **Declarative submissions only:** `agent.yaml`, prompts, sampling config, optional LoRA adapters (`.safetensors`), and ADK *skills* (`SKILL.md` + Python scripts). No arbitrary Python entrypoint; allowed file types `.yaml .yml .md .txt .py .json .safetensors`; <3 GiB.
- **Budgets:** per-task time / tool calls / turns from `eval_config.yaml`; ADK compacts the conversation once it exceeds 14,336 tokens.
- **Runtime:** ≈120 hidden tasks in 12 hours; if tasks run serially that is ≤6 minutes per task on average.

- **Rules** (`reports/rules.md`): external data allowed if public and free; open-source code must be OSI-licensed without commercial restrictions; winners release code (Apache-2.0); 1 submission/day, 2 final selections; no private sharing outside the team.

3 Data

3.1 Audit

repo	tasks	median graph nodes	median files	median hunks	median +lines	median −lines
fastapi	67	3931	1	2	11	2
rich	48	1970	1	2	5	3
requests	13	733	1	1	3	1
httpx	1	740	7	18	57	22

Table 2: Data audit (`src/dataio/audit.py`). Gold patches are small and never touch tests.

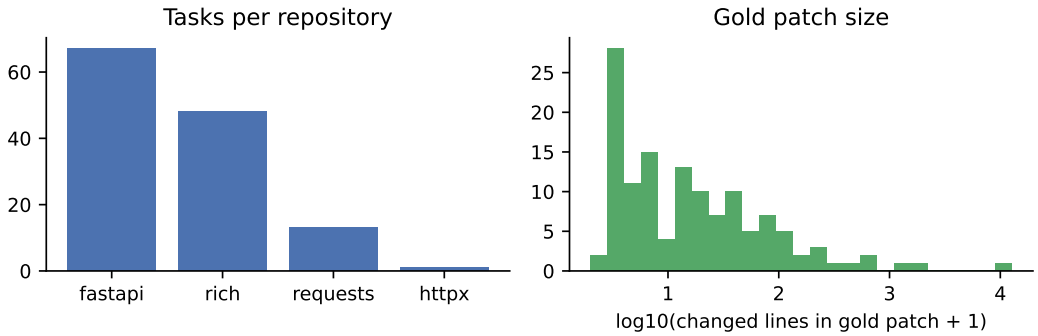


Figure 1: Tasks per repository and gold-patch size distribution.

Each task ships a code graph (JSON, one per base commit) and node embeddings (256-dimensional, unknown encoder). Graph nodes are {`id`, `name`, `text`} with dotted names such as `requests.utils.get_encoding_from_headers`; there is a single edge type `calls` which, as we discovered while building our own graph (§9.2), stores every relation in *both* directions and also encodes class↔method containment. Nodes carry no file paths or line numbers.

Intuition. With one repository dominating (fastapi 52%) and httpx contributing a single task, leave-one-repository-out (LORO) validation effectively has three folds. Any learned component sees only two or three repositories during training — the root cause of several later findings.

4 Infrastructure

4.1 Three Python environments

The harness wheels target CPython 3.12 (vLLM, torch 2.10/CUDA 12.8), while the scorer's sandbox image is `python:3.13-slim`. We therefore keep: `.venv` (3.13, research code), `.venv-harness` (3.12, model server only) and `.venv-eval` (3.13 on a clean standalone interpreter, harness without torch). The sandbox inherits the interpreter's site-packages, so a clean base interpreter is essential — the system Python carried 706 Debian packages that would have leaked into the sandbox.

4.2 A Docker-free reproduction of the scorer

No container runtime exists on the server, so evaluation uses the harness's subprocess sandbox. Out of the box it skips dependency installation (on Kaggle the notebook image already has everything). We patched it (`src/eval/harness_patches.py`) to resolve each task's dependencies with `pip` from the offline wheel directory *under the repository's own version constraints*. Installing the newest version of every wheel (what the Docker path does) shadows the repository's own package and breaks older commits (e.g. `starlette 1.6`

removed `on_startup`). We reconstructed the scorer image's test-only plugins from evidence in the harness (`pytest-httpbin`, `inline-snapshot`, `dirty-equals`, …).

Validity. A task is *locally valid* iff the gold patch resolves it and an empty patch does not:

$$V = \{t : \ R(t, p_t^\star) = 1 \ \wedge \ R(t, \varnothing) = 0\}.$$

repo	valid	total	main reasons for exclusion
fastapi	58	67	dependency drift; 2 tasks pass with an empty patch
rich	39	48	newer pygments / markdown-it change rendered output
requests	9	13	idna validation drift, one SSL-certificate failure
httpx	1	1	–
total	**107**	129	

Table 3: Local validity (`data/derived/valid_tasks.txt`).

4.3 Serving a 31B model on two shared 24 GB GPUs

Memory arithmetic. One decoder layer has attention projections $q : 5376 \to 8192$, $k, v : 5376 \to 4096$, $o : 8192 \to 5376$ (sliding-window layers) and an MLP of three $5376 \leftrightarrow 21504$ matrices:

$$P_{\text{layer}} \approx 44.0\text{M} + 2 \cdot 22.0\text{M} + 44.0\text{M} + 3 \cdot 115.6\text{M} \approx 479\text{M parameters.}$$

W4A16 with group size 32 stores 4 bits per weight plus a 16-bit scale per 32 weights, i.e. $0.5 + 2/32 = 0.5625$ bytes/parameter, so ≈ 269 MB ≈ 0.25 GiB per layer — matching the measured 0.27 GiB/layer (16.58 GiB for 53 layers + embedding vs 16.86 GiB for 54). The embedding is $262{,}144 \times 5{,}376$ in bf16 = 2.62 GiB, and with pipeline parallelism the last stage needs its own copy of the tied output head. Tensor parallelism over two GPUs needs ≈12 GiB of weights per GPU plus KV cache; the second GPU never had more than 9.2 GiB free (another user's server holds 15 GiB).

KV cache. Gemma-4-31B is hybrid: 50 sliding-window layers (window 1024) and 10 global layers (4 KV heads, head dim 512). The KV bytes per token of context come almost entirely from global layers: $10 \times 4 \times 512 \times 2\,(\text{K, V}) \times 2\,\text{bytes} = 80$ KiB. vLLM's measured requirement for one full-length request fits a line

$$\text{KV}(L) \approx a + bL, \qquad b \approx 70 \text{ KiB/token}, \ a \approx 2.2 \text{ GiB}$$

from the points (20,480, 3.55), (24,576, 3.81), (32,768, 4.37) GiB — b is consistent with the 80 KiB estimate; a is the fixed sliding-window/prefill-chunk allocation.

Solution: an uneven pipeline split. Pipeline parallelism (PP=2) with `VLLM_PP_LAYER_PARTITION=52,8` places 52 layers on GPU0 and 8 layers + output head on GPU1. Four local patches to vLLM were needed (`scripts/patch_vllm_gemma4_pp.py`):

1. Gemma-4's PP path called `IntermediateTensors.get()` (no such method) and sent `None` residual/per-layer tensors between stages. Each layer ignores its incoming residual, so sending zeros is numerically identical.
2. A per-rank KV budget (`GEMMA_RL_KV_GIB_PER_RANK`), because one global budget cannot fit both an almost-full and an almost-empty GPU.
3. Clamping `max_tokens` to the remaining context instead of rejecting requests (the scorer's 32k context leaves room that our 20k context does not).

plus the Kaggle-wheelhouse vLLM build (PyPI's build lacks LoRA support for Gemma-4), FastAPI <0.130, and `--reasoning-config` with Gemma's channel tokens. Concurrency >1 triggered a PP scheduler stall after preemption, so the server runs one sequence at a time (`MAX_SEQS=1`). Throughput: 21.5 tokens/s per stream. The working configuration is `scripts/start_server.sh`.

4.4 Evaluation on the scorer's hardware

Kaggle notebooks run on the same 4×L4 machine type as the scorer. `scripts/kaggle_eval/` builds a private kernel per experiment: it installs the official wheelhouse exactly like the organizers' starter notebook, serves the model with the scorer's vLLM settings, runs the official `Evaluator` with the scorer's compaction settings (interval 5, overlap 2, threshold 14,336, retention 5) at concurrency 4, and downloads results, patches, test logs and full traces. Two fixes were required: a private dataset with the test-only packages the Kaggle image lacks (without it every fastapi test fails on `inline_snapshot`), and an in-kernel *capture proxy* that records every LLM request/response for supervised training data.

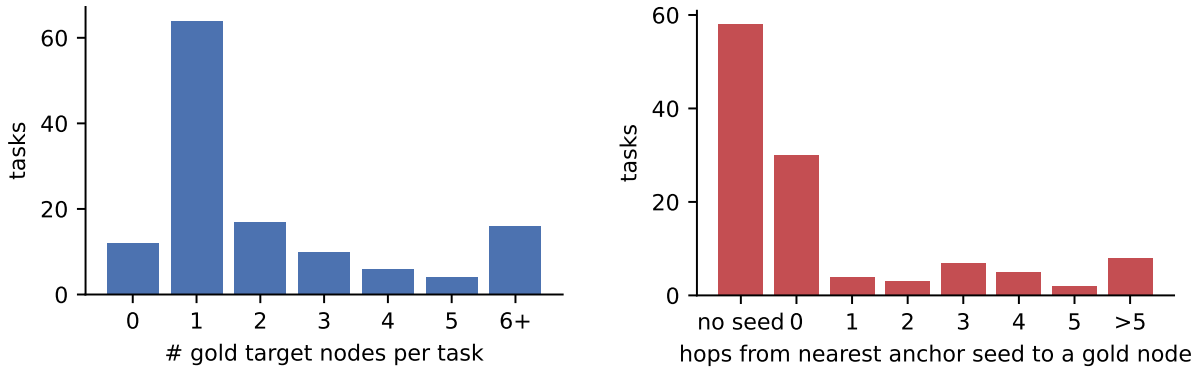
5 Fault localization research

5.1 Ground-truth labels from gold patches (B1)

Localization has free supervision. For a gold patch, let $\mathcal{L}$ be the changed old-file line numbers (removed lines, and for pure insertions the line above the insertion point). Parsing the file at `base_commit` with Python's `ast` yields the qualified-name spans $\{(q, \ell_{\text{start}}, \ell_{\text{end}})\}$ of all definitions (decorators included). The module prefix m of a file is the dotted suffix of its path that maximizes the number of definitions found in the graph. A changed line ℓ maps to the innermost span containing it whose name $m.q$ exists in the graph:

$$\text{target}(\ell) = \arg \min_{(q,s,e):\ s \le \ell \le e,\ m.q \in G} (e - s).$$

Results: 117/129 tasks (91%) have ≥1 target node; unmapped lines are mostly module-level (imports, constants: 1,127) or functions missing from the provided graph (92).



(a) Gold nodes per task.

(b) Graph distance from issue anchors to gold.

Figure 2: Label statistics. (b) For 58 of 117 tasks no issue identifier selects a specific node (anchor matches >25 nodes or none); for 30 an anchor hits a gold node directly; the rest need 1–5 hops.

Intuition. Half the issues contain no selective identifier, so any graph traversal must start from lexical retrieval (BM25) rather than from names in the issue; and when a name is present, it is very often the answer itself (distance 0). The remaining tasks, where the answer is a few hops from a named symbol, are exactly where learned traversal could help.

5.2 Anchors

From the issue text we extract: identifiers inside backticks; snake_case / CamelCase / dotted tokens; file paths; traceback frames. An anchor's weight is its inverse document frequency over node short names, $w(a) = \log(1 + N/n_a)$ where n_a is the number of nodes with that short name; anchors matching ≤25 nodes become graph *seeds*.

5.3 R1: BM25

Nodes are documents d whose tokens are the name tokens (camel/snake-split, weighted ×3) plus the tokenized source text. For query tokens q:

$$\text{BM25}(d,q) = \sum_{w \in q} \text{idf}(w)\, \frac{f(w,d)\,(k_1+1)}{f(w,d) + k_1\big(1 - b + b\,\frac{|d|}{\text{avgdl}}\big)} \cdot \min(\text{qtf}_w, 3), \qquad \text{idf}(w) = \log\Big(1 + \frac{N - n_w + 0.5}{n_w + 0.5}\Big),$$

with $k_1 = 1.2$, $b = 0.75$. *Intuition:* term frequency saturates (a word appearing 20 times is not 20× more relevant), rare words count more, long nodes are normalized.

5.4 R2: embedding pseudo-query

The 256-d node embeddings come from an unknown encoder, so the issue cannot be embedded. We form a pseudo-query from the seeds S (or BM25's top-5 when there are none): $\mathbf{c} = \frac{\sum_{i \in S} \mathbf{e}_i}{\|\sum_{i \in S} \mathbf{e}_i\|}$ and score $s_i = \mathbf{e}_i^\top \mathbf{c}$ (embeddings ℓ_2-normalized). This mirrors the harness tool `search_similar_code`, which also only accepts symbol names.

5.5 R3: learning to rank (LambdaMART)

Every non-test node gets a 29-dimensional feature vector: BM25 (raw, max-normalized, rank percentile, name-only), embedding cosine/rank/max-to-seed, anchor matches (idf-weighted short name, backtick, dotted-path, traceback, mentioned module, parent/child anchors), graph distance to seeds and to BM25's top-5, degree, node kind, privacy/dunder flags, depth, in-package flag, text length, name–query overlap and the best BM25 score in the same module. LightGBM's LambdaRank optimizes NDCG directly: for a pair (i, j) with $y_i > y_j$ in the same query, the gradient on s_i is

$$\lambda_{ij} = \frac{-\sigma}{1 + e^{\sigma(s_i - s_j)}} \left|\Delta\mathrm{NDCG}_{ij}\right|, \qquad \lambda_i = \sum_{j : y_i > y_j} \lambda_{ij} - \sum_{j : y_j > y_i} \lambda_{ji},$$

i.e. a RankNet pairwise logistic gradient re-weighted by how much swapping i and j would change NDCG. Training uses all positives plus hard negatives (top-100 by BM25, embedding and anchor score) and 200 random negatives per task; 300 rounds, 15 leaves, learning rate 0.05, 3 seeds; folds are LORO.

5.6 R3+G: graph smoothing

With z_i the per-task z-scored ranker output and $\mathcal{N}(i)$ the undirected neighbours,

$$s'_i = z_i + \alpha \max_{j \in \mathcal{N}(i)} z_j,$$

with $\alpha \in \{0, 0.1, \ldots, 0.75\}$ chosen per fold on the *other* folds' out-of-fold predictions.

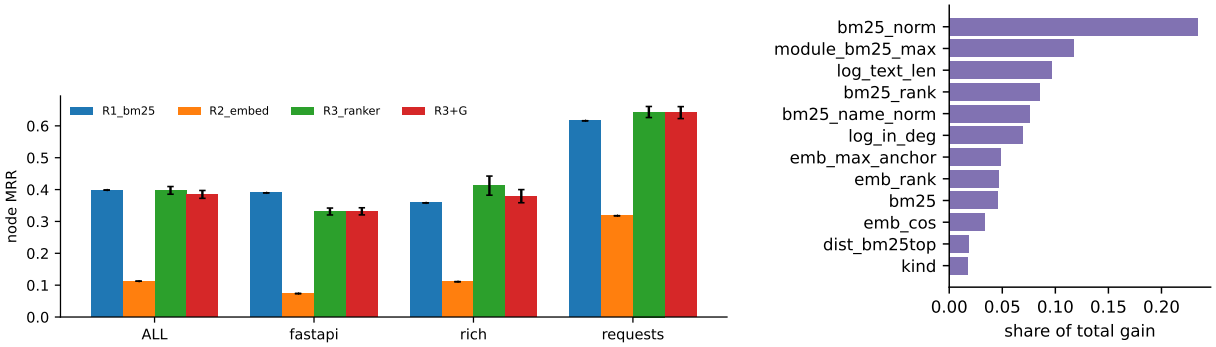
5.7 Metrics

For a ranking π of candidates and gold set T: Recall@$k = \frac{|\pi_{1..k} \cap T|}{|T|}$, Hit@$k = \mathbb{1}[\pi_{1..k} \cap T \neq \varnothing]$, MRR = 1/rank of the first gold, nDCG@10 $= \frac{\sum_{r \le 10, \pi_r \in T} 1/\log_2(r+1)}{\sum_{r=1}^{\min(|T|, 10)} 1/\log_2(r+1)}$. File-level Hit@$k$ deduplicates the node ranking into a module ranking.

5.8 Results

fold	system	R@1	R@5	R@10	Hit@5	MRR	nDCG@10	File@1
ALL (117)	R1 BM25	.183	.384	.457	.513	**.398**	**.373**	.496
	R2 embedding	.047	.099	.116	.154	.112	.091	.248
	R3 ranker	.185	.369	.444	.501	.397	.358	**.501**
	R3+G	.177	.373	.432	.501	.385	.350	.487
fastapi (60)	R1 / R3	.163/.136	.354/.303	.448/.368	.483/.428	**.390**/.331	.371/.305	.433/.367
rich (45)	R1 / R3	.148/.164	.332/.363	.387/.462	.467/.511	.358/**.412**	.305/.354	.533/.615
requests (11)	R1 / R3	.455/.545	.788/.778	.818/.818	.818/.818	.616/**.643**	.663/.681	.636/.727

Table 4: LORO localization (`reports/localization_baselines.md`); R3 values are means over 3 seeds (std ≤0.04).



(a) Node MRR per fold.

(b) R3 feature importance (gain).

Figure 3: Localization baselines.

Finding. BM25 is the baseline to beat. The supervised ranker matches it overall, wins on rich and requests, and loses on fastapi when trained on only two other repositories; its most important features are themselves lexical. Graph smoothing does not help. The bottleneck is *training data*, not model capacity — which motivates the mined and synthetic data of §9.

5.9 The deployed locator skill

`skills/repo-tools/scripts/locate.py` (pure standard library, runs inside the sandbox) parses every non-test Python file with `ast`, scores each definition with BM25 over name tokens (×3) + body (first 120 lines) + path tokens, then adds anchor bonuses

$$s_i = \frac{\mathrm{BM25}_i}{\max_j \mathrm{BM25}_j} + \underbrace{\frac{0.6}{\sqrt{c_i}}\mathbb{1}[\text{backtick}]}_{\text{or } 0.3 \text{ for identifiers}} + 0.4\,\mathbb{1}[\text{dotted match}] + 0.25\,\mathbb{1}[\text{file mentioned}] + \lambda_{\text{path}} \frac{\sum_{w \in P_i \cap q} \mathrm{idf}_f(w)}{\sum_{w \in P_i} \mathrm{idf}_f(w)},$$

where c_i counts definitions sharing the short name and P_i are the tokens of the file path (generic tokens such as `utils` removed), with idf computed over files. The last term captures concept-to-file mappings ("OpenAPI" → `openapi/utils.py`).

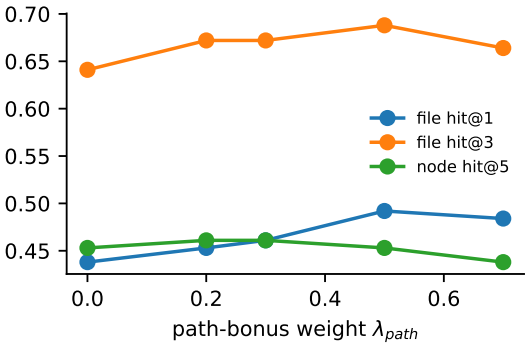


Figure 4: Sweep of λ_{path} on 128 tasks; $\lambda_{\text{path}} = 0.5$ is used (file hit@1 0.438→0.492, hit@3 0.641→0.688).

6 The agent (scaffold) and end-to-end experiments

6.1 How the harness drives an agent

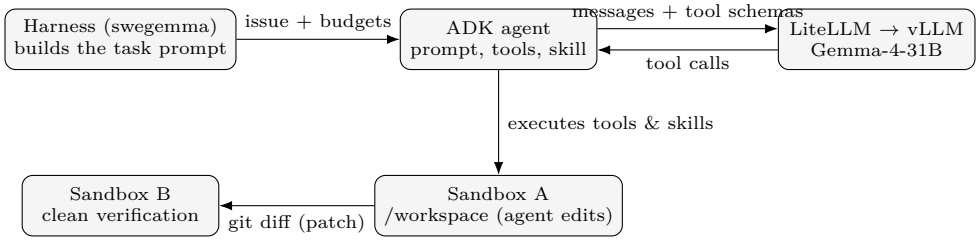


Figure 5: Phase 1 runs the agent in sandbox A until `submit_patch`, a budget limit or 3 nudges; phase 2 verifies the diff in a clean sandbox B.

Tools: `run_command`, `read_file` (150 lines / 10k chars), `edit_file` (3-tier fuzzy replacement), `write_file`, `get_status`, `submit_patch`, three graph tools, and the ADK skill tools (`list_skills`, `load_skill`, `run_skill_script`). Skill calls do not count against the tool budget. Our `repo-tools` skill provides `locate.py`, `show.py` (definition with line numbers), `tests.py` (tests mentioning a name) and `check.py` (clean-up + compile + targeted pytest).

6.2 Configurations

C1	Structured six-step prompt (anchors → locate → read → edit → verify → submit), repo-tools skill, thinking budget 2048, 8 min / 45 tool calls.
C2	+ explicit tool list, JSON skill-call templates, "edit by the 10th call" rule.
C3	Skill instructions as plain text (no backticks), fixed argument order.
C4	+ `check.py` automatically reverts test/config edits and deletes scratch files; time rule.
C5a / C5b	C4 with thinking budget 1024 / thinking disabled.
C6a	Thinking disabled, 100 tool calls / 150 turns, "never submit an empty patch". **Frozen.**
C6b	Thinking 512, 80 tool calls, same rules.

Table 5: Scaffold versions (`submissions/`).

6.3 Results on the scorer's hardware

All configurations were run on the same 30 locally valid tasks on Kaggle 4×L4 with the scorer's settings.

config	resolved	mean s/task	timeouts	test-patch failures	right file, wrong fix	wrong location	no patch
C1	8	415	16	3	10	5	1
C2	8	415	13	4	10	4	0
C3	**10**	428	14	1	11	3	1
C4	9	409	17	**0**	8	3	0
C5a	9	401	14	0	11	3	1
C5b	7	**186**	2	1	6	2	13
C6a	9	270	4	1	6	3	7
C6b	9	420	16	0	10	2	0

Table 6: End-to-end results (`reports/triage_*.md`). "Timeouts" counts sessions stopped by the time budget (their partial patch is still scored).

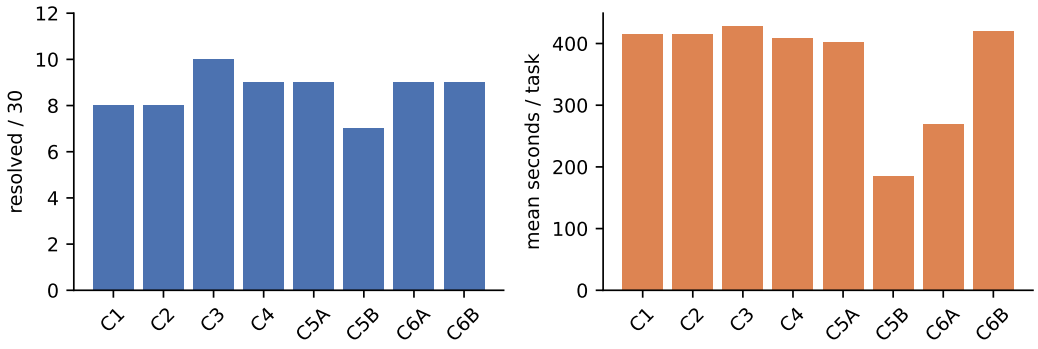


Figure 6: Resolved tasks and mean time per task.

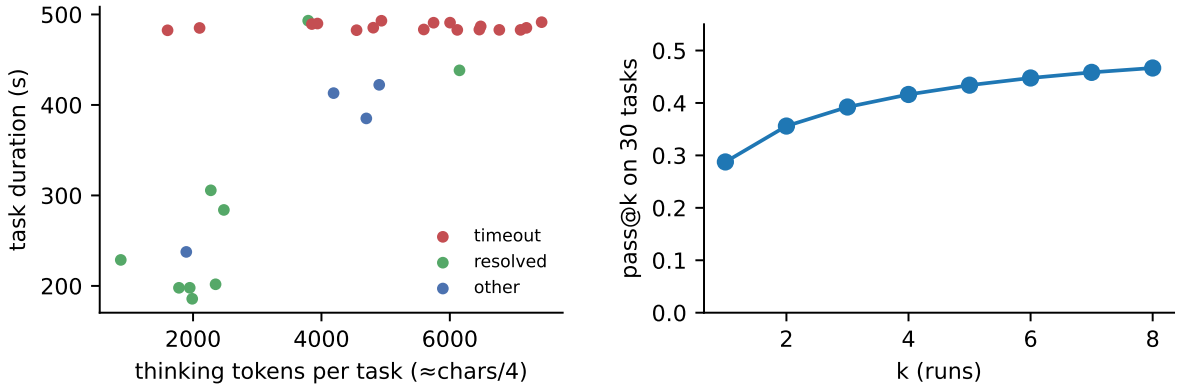
6.4 Four failure mechanisms we found and fixed

1. Malformed skill calls. Gemma emits tool calls in a native syntax where string values are delimited by a special token `<|"|>`. When the prompt showed paths inside markdown backticks, the model sometimes closed a string with a backtick, producing `"file_path": "scripts/locate.py`,skill_name:"` and losing the required argument (C2: 30 of 35 locate calls malformed). Writing all skill instructions as plain text (C3) removed the problem.

2. The test-patch trap. Verification resets protected test files and then applies the hidden test patch. We reproduced locally that if the agent edited a test file that the hidden patch also touches, the hidden patch fails to apply and the task scores 0 regardless of the fix (3–4 tasks per run in C1/C2). `check.py` now reverts such edits automatically (0 failures in C4).

3. Thinking-driven timeouts. Thinking volume and task duration correlate strongly ($r = 0.72$, Fig. 7a); single thoughts reached the 2048-token cap, and at ≈20 tokens/s each such thought costs ≈100 s of an 8-minute budget.

4. Budget-bound empty submissions. Without thinking the agent is 2.2× faster but makes ≈40 shallow calls in ≈90 s, hits the 45-call budget, and submits nothing (13/30 in C5b). C6a raises the budget to 100 calls and forbids empty submissions: same accuracy as C4 at 66% of the time.



(a) C4: thinking vs duration.

(b) pass@k across the 8 runs.

Figure 7: Diagnostics.

6.5 How much of a difference is real? Noise, pass@k and stochastic tasks

With per-task success probability $p \approx 0.3$ and $n = 30$ tasks the binomial standard error is $\sqrt{p(1-p)/n} \approx 0.084$, i.e. ±2.5 tasks; with $n = 107$ it is 0.044 (±4.7 tasks). The observed run-to-run standard deviation across configurations was 0.024. Differences of one or two tasks on 30 are therefore not evidence. The plan's protocol (§7) uses 107 tasks, ≥4 runs and paired tests.

Treating the 8 runs as samples, the unbiased pass@k estimator is

$$\text{pass@}k = \mathbb{E}_t\Big[1 - \binom{n - c_t}{k} \Big/ \binom{n}{k}\Big],$$

where c_t is the number of runs that solved t. It rises from 0.29 (k=1) to **0.467** (k=8): 14 distinct tasks are solvable; 4 are solved in all 8 runs ("stable") and 10 only in some ("stochastic").

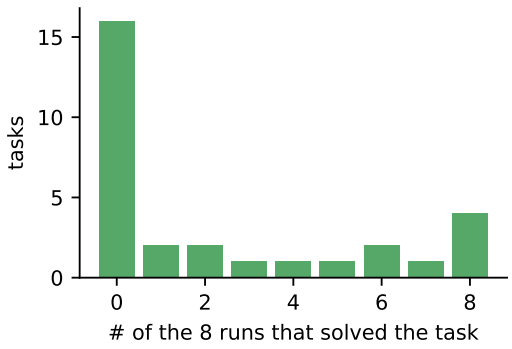


Figure 8: How many of the 8 runs solved each task.

Intuition. If the model were trained to make the choices it makes in its successful runs, single-run accuracy should move from pass@1 towards pass@k. That is precisely what rejection-sampling fine-tuning does (§8).

7 Measurement protocol

Every competition-facing claim uses: the 107 valid tasks; ≥4 runs per configuration (≥2 for the organizers' sample baseline B0); pass@1 reported as mean±std across runs, pass@k, numbers of stable and stochastic tasks, time per task, tool-error rate and tokens per task (`src/eval/scoreboard.py`). A difference counts only if it exceeds twice the larger standard deviation or passes a paired test on per-task majority outcomes (exact McNemar: with b tasks only system A solves and c only B, $p = 2\sum_{j=0}^{\min(b,c)} \binom{b+c}{j} 2^{-(b+c)}$). Anything trained is evaluated LORO. Public-leaderboard scores (≈60 tasks, 1 task ≈1.7%) are never a selection criterion. On the pilot data (4 configurations, 30 tasks) the scoreboard gives pass@1 0.300 ± 0.024, pass@4 0.467, 5 stable, 9 stochastic. The reference runs (C6a×4, B0×2, 107 tasks) are queued on Kaggle.

8 Fine-tuning groundwork

8.1 LoRA on the quantized checkpoint

LoRA adds a low-rank update to a frozen weight $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$:

$$h = Wx + \frac{\alpha}{r} BAx, \qquad A \in \mathbb{R}^{r \times d_{\text{in}}},\ B \in \mathbb{R}^{d_{\text{out}} \times r},$$

so only $r(d_{\text{in}} + d_{\text{out}})$ parameters per matrix are trained, and the 4-bit base weights never change. Gemma-4's global-attention layers have different shapes ($q : 5376 \to 16384$, $k : 5376 \to 2048$, no v because keys double as values), so adapter shapes are read from the checkpoint (`src/sft/make_smoke_adapter.py`). For $r = 16$ on all seven projections:

$$50 \times 2.03\text{M} + 10 \times 2.11\text{M} = 122.4\text{M parameters} = 233.5\ \text{MiB (bf16)}, \quad 820\ \text{tensors}.$$

The smoke test (random A, $B \sim \mathcal{N}(0, 0.02^2)$) loads on the QAT checkpoint, and greedy outputs diverge from the base model mid-sentence ("added functionality" → "enhanced functionality"), proving the adapter is applied; decoding slows from 21.5 to 15 tokens/s locally (eager mode). The Kaggle-side test is queued.

8.2 Supervised objective

For a token sequence $x_{1:T}$ with loss mask $m_t \in \{0, 1\}$ (1 exactly on tokens the model generated):

$$\mathcal{L}(\theta) = -\frac{1}{\sum_t m_t} \sum_{t=1}^{T} m_t \log p_\theta(x_t \mid x_{<t}).$$

Rejection-sampling fine-tuning (RFT/RAFT) samples n trajectories per task at temperature 0.6–0.8, keeps those whose patches pass the tests, and minimizes $\mathcal{L}$ on them. It is the special case of reward-weighted regression with a binary reward: maximizing $\mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)]$ by repeatedly fitting π_θ to the reward-filtered distribution $\propto \pi_{\theta_{\text{old}}}(\tau)R(\tau)$. Expert iteration repeats this with the improved model.

8.3 Rendering trajectories in the exact training format

Training data is only useful if it matches, token for token, what the model sees at inference. We recorded the real requests with a logging proxy and established:

- The system prompt is our instruction *plus* ADK's agent-identity and skill-toolset text; the user message is the harness prompt; tool calls are assistant messages with JSON-string arguments; tool results are `tool` messages; 12 tool schemas.
- vLLM parses argument strings into dicts before templating, producing Gemma's native call syntax:

```
<|tool_call>call:read_file{end_line:650,filepath:<|"|>src/requests/models.py<|"|>,start_line:574}<
    tool_call|>
```

- After tool calls the model stops by emitting the token `<|tool_response>` (id 50); text answers end with `<turn|>` (id 106). Both are part of the training target.
- With thinking disabled, the first generation of a model turn sees an empty thought channel that history renders never contain; we use the history format:

```
<|channel>thought\n<channel|>
```

Round trip on a captured run: 15/16 of vLLM's prompts are exact token prefixes of our training sequence (the 16th differs only by those 3 empty-thought tokens), all 16 assistant spans are trained, and vLLM's own Gemma-4 tool parser recovers the original calls from the trained spans.

8.4 Context compaction and segmentation

ADK replaces old events with a summary once the context exceeds 14,336 tokens. 35 of 46 passing pilot trajectories are longer than that (Fig. 9), so the full trace is *not* what the model saw. The dataset builder (`src/sft/build_dataset.py`) therefore uses the recorded per-call prompts: consecutive calls belong to one *segment* iff

$$\text{prompt}_{k+1}[1{:}|\text{prompt}_k| + 1] = \text{prompt}_k \parallel \text{response}_k,$$

and each segment becomes one sequence whose mask covers the assistant turns generated within it.

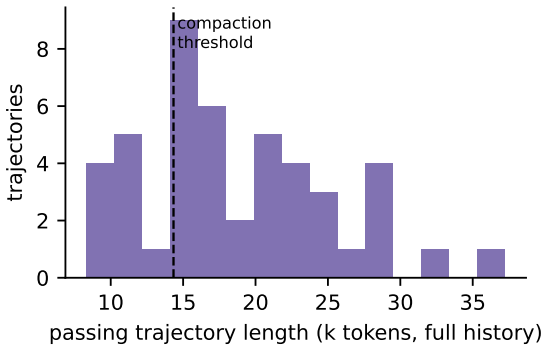


Figure 9: Lengths of passing trajectories vs the compaction threshold.

9 New training data

9.1 Tasks mined from git history (B4)

Snapshots contain the full history before each base commit. A candidate is a non-merge commit touching both source and test Python files, ≤5 files and ≤300 changed lines; its diff is split into `patch` (source) and `test_patch` (tests); the base is its parent; the commit message is the problem statement. *Leakage guard:* snapshot histories were rebuilt with `git fast-export`, so evaluation base SHAs are unresolvable; we keep only commits older than the earliest evaluation PR minus 60 days and drop any commit whose hunks touch a function changed by any evaluation gold patch.

repo	commits before cutoff	candidates	verified	dominant failure
rich	2902	260	**62**	tests already pass without the fix (22)
fastapi	5881	55	4	96% of commits are docs/translations
requests	4654	280	0	`import cgi` (removed in Python 3.13), old urllib3
total		595	**66**	

Table 7: Mining yield (`reports/b4_mining.md`). 48/66 have short commit messages and will receive generated issue text.

9.2 Our own code-graph builder

`src/graphbuild/build.py` reproduces the organizers' conventions: dotted names relative to the repository root (`src/`, `docs_src/` stripped, `__init__` collapsed); nested definitions named under the nearest enclosing class or module; class text = header + docstring + method stubs; `calls` edges for containment and for calls resolvable through imports or same-module definitions, stored in both directions.

repo	node recall	node precision	edge recall	edge precision
fastapi	1.000	0.828	0.731	0.644
rich	1.000	0.999	0.919	0.950
httpx	1.000	0.881	0.758	0.779
requests	0.787†	0.787	0.695	0.733

Table 8: Parity with the provided graphs. †All requests mismatches are test-class members named differently; non-test parity is 1.0.

9.3 Synthetic localization bugs (B5)

From 22 non-evaluation packages in the competition wheels (starlette, pydantic, urllib3, sqlalchemy, …) we sample functions and apply one AST mutation at one site, replacing only that function's source span.

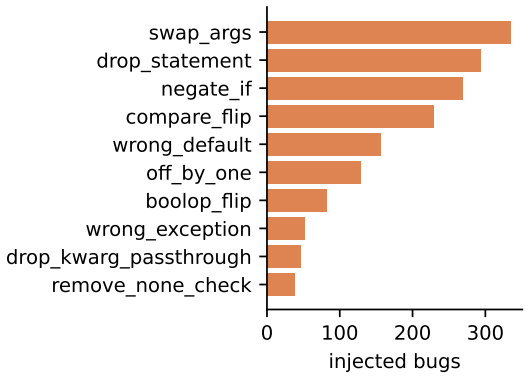


Figure 10: 1,628 injected bugs by operator.

Gemma writes each issue as a user would (public-API reproduction, expected vs actual, 40% with a traceback, never naming the internal function). Because real issues mention their target with some rate ρ_{real}, synthetic issues are subsampled so that the mention rate matches: keeping all U non-mentioning issues and $k =$

$\frac{\rho_{\text{real}}}{1-\rho_{\text{real}}}U$ mentioning ones gives $\frac{k}{U+k} = \rho_{\text{real}}$. The distribution check (`src/retrieval/run_synth_check.py`) retrains the ranker on real+synthetic data without embedding features and compares with real-only training on each held-out real repository; it decides Gate B. Issue generation is running (about 20 s per issue on the local server).

10 The reinforcement-learning traverser (designed, data-gated)

10.1 MDP

An episode is one task. The state $s_h = (q, F_h, M_h)$ holds the issue features q, the frontier F_h of visited nodes with their feature vectors (shared with R3), and the marked set M_h. Actions are $\textsc{Expand}(v, \text{dir})$ for $v \in F_h$ and $\text{dir} \in \{\text{out}, \text{in}\}$ (the graph has one edge type; we also distinguish containment), $\textsc{Mark}(v)$ and $\textsc{Stop}$; horizon $H \approx 20$. The initial frontier is the seed set S or BM25's top-k. The terminal reward favours recall,

$$R = F_\beta(M_H, T) - \lambda H, \qquad F_\beta = \frac{(1+\beta^2)PR}{\beta^2 P + R},\ \beta > 1,$$

optionally with potential-based shaping $r_h + \gamma\Phi(s_{h+1}) - \Phi(s_h)$, $\Phi(s) = -\min_{v\in F, u\in T} d(v,u)$, which provably leaves the optimal policy unchanged.

10.2 Policy and learning

A pointer-style policy scores every candidate action with one shared network over $[\phi(q)\|\phi(v)\|e_{\text{dir}}]$, so a variable-size frontier needs no fixed action dimension:

$$\pi_\theta(a \mid s) = \frac{\exp f_\theta(a, s)}{\sum_{a' \in \mathcal{A}(s)} \exp f_\theta(a', s)}.$$

Behaviour cloning on shortest seed→target paths (expert actions $a_h^\star$) minimizes

$$\mathcal{L}_{\text{BC}}(\theta) = -\sum_h \log \pi_\theta(a_h^\star \mid s_h).$$

REINFORCE with a learned baseline b_ψ then follows

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\Big[\sum_h \nabla_\theta \log \pi_\theta(a_h \mid s_h)\,\big(G_h - b_\psi(s_h)\big)\Big], \qquad G_h = \sum_{h'\ge h} r_{h'},$$

and PPO's clipped surrogate is used only if variance is a problem:

$$J_{\text{PPO}}(\theta) = \mathbb{E}\big[\min\big(\rho_h \hat{A}_h,\ \text{clip}(\rho_h, 1-\epsilon, 1+\epsilon)\hat{A}_h\big)\big], \qquad \rho_h = \frac{\pi_\theta(a_h \mid s_h)}{\pi_{\theta_{\text{old}}}(a_h \mid s_h)}.$$

Intuition: BC teaches the policy to walk towards the answer the way a shortest path would; policy gradients then optimize the actual objective (recall of the final marked set under a step cost), letting the policy learn when to stop and when exploring a neighbour is worth a step.

10.3 Why it is gated, and how it would ship

RL must beat the best non-RL retriever on held-out repositories; with three real training repositories even the supervised ranker does not beat BM25, so RL waits for the synthetic/external data. Deployment (Option A, `reports/optionAB.md`): a pure-Python policy with JSON weights inside `locate.py`, building its graph from `/workspace` with our builder (the organizers' graphs are not reachable from the sandbox).

11 Lessons learned

- **Measure the harness, not your assumptions.** The test-patch trap, malformed tool calls and context compaction were all invisible until we read raw logs and captured requests.
- **Small evaluation sets lie.** On 30 tasks, ±2.5 tasks is one standard error; the protocol now requires 107 tasks, multiple runs and paired tests.

- **Time is a first-class metric.** Thinking budget, tool budget and per-task time interact; the fastest configuration with equal accuracy (C6a) is the safest under a 12-hour cap.
- **Prompt formatting is part of the model interface.** Markdown in instructions leaked into structured tool calls.
- **Learned components need diverse data.** Three repositories are not enough to beat BM25; mined and synthetic data are prerequisites, not extras.
- **Shared infrastructure needs defensive engineering**: uneven pipeline splits, per-rank memory budgets, port collisions with other users' services, and self-matching `pkill` patterns all cost real time.

12 Status and next steps

Done	Data audit; local harness (107 valid); serving; Kaggle pipeline with capture; B1; B2; locator skill; C1–C6; freeze of C6a; rules; LoRA smoke (local); scoreboard; E0 formatting; B4 mining; graph builder; B5 injection; Option A/B memo.
Running / queued	Kaggle: LoRA smoke, C6a×4, B0×2 (reference measurement and first SFT data); synthetic issue generation.
Next	P5 triage of stochastic tasks (decides whether localization matters for the competition); B5 distribution check (Gate B); E1 rejection-sampling SFT on captured successes + mined tasks, LORO, on rented 80 GB GPU; E2 expert iteration; C6a submission.
Risks	Kaggle GPU availability; QAT+LoRA quality; 12-hour runtime if the scorer runs tasks serially; synthetic-issue distribution shift.

A Where things are

`src/dataio/`	loaders, audit
`src/groundtruth/map_patch.py`	B1 patch→node labels
`src/retrieval/`	text/anchors, context (BM25, graph index), features, baselines, synthetic check
`src/eval/`	harness patches, gold/empty verification, valid set, scorer-faithful runner, triage, scoreboard, locator evaluation, validator
`src/sft/`	smoke adapter, capture proxy, exact-format rendering, dataset builder
`src/mining/`, `src/synth/`, `src/graphbuild/`	B4 mining, B5 injection and issues, graph builder
`submissions/`	B0, C1–C6 scaffolds, LoRA smoke
`scripts/`	serving (`start_server.sh`, patches), local eval, Kaggle kernels/queue, packaging
`reports/`	all result tables, memos, this report

B Reproducing the main results

```
.venv/bin/python -m src.dataio.audit # data audit
.venv-eval/bin/python -m src.eval.verify_gold --mode gold # + --mode empty; then src.eval.valid_set
.venv/bin/python -m src.groundtruth.map_patch # B1
.venv/bin/python -m src.retrieval.run_baselines --seeds 3 # B2 table
.venv/bin/python -m src.eval.eval_locate_skill # locator skill accuracy
scripts/start_server.sh # local model server (port 8011)
.venv/bin/python scripts/kaggle_eval/make_kernel.py --submission submissions/c6a --run NAME --n 30 --push
.venv/bin/python -m src.eval.triage results/kaggle/NAME/results/NAME
.venv-harness/bin/python -m src.sft.format --roundtrip results/capture.jsonl --trace <trace> --scaffold c6a
```